ENG3091 APSE: Assignment 1 Group Report

William Chambers[1], Joel Duncan[2], and Saad Aziz[3]

[1]2906463C
[2]2734528D
[3]2917444A

March 30, 2026

Abstract

This report details the implementation of foundational programming and machine learning concepts in C++, transitioning from Python-based prototypes to statically typed, compiled software. The project is divided into two primary tasks. Task 1 focuses on algorithmic translation and game logic, featuring solvers for non-linear equations (Fixed-Point and Newton-Raphson methods), an interactive Rogue-like grid navigation system, and a console-based Tic-Tac-Toe game. Task 2 explores the mathematical mechanics of Machine Learning by building a Linear Regression engine from scratch. Datasets were programmatically generated with synthetic noise, and model parameters were optimized using both analytical Normal Equations and iterative Gradient Descent. The results demonstrate the efficiency of C++ in handling mathematical simulations and algorithmic logic.

1 Introduction & Setup

The primary objective of this assignment is to develop an understanding of C++ memory management, standard libraries, and control structures, culminating in the development of a functional machine learning model. To ensure reproducibility, this section outlines the dependencies and exact compilation steps required to execute the source code.

1.1 Dependencies for task 1

To successfully compile and run the programs, the following dependencies are required:

- A modern C++ compiler (e.g., GCC or Clang) supporting the **C++20** standard, as features like `<ranges>` are utilized.
- For the Newton-Raphson visualizer (`Newton_Raphson.cpp`), **Python 3** and **Matplotlib** must be installed, along with the Python development headers (`python3-dev`), to support the `matplotlibcpp.h` wrapper.

1.2 Compilation and Execution Guide for Task 1

Assuming a standard Linux/Unix environment with `g++` installed, the following bash commands can be used to compile and run each program.

```
# 1. Fixed Point Iteration
g++ -std=c++20 Fixed_Point.cpp -o Fixed_Point
./Fixed_Point

# 2. Newton-Raphson (Requires Python linking)
# Note: Python version (e.g., python3.10) may need adjustment based on the local system.
g++ -std=c++20 Newton_Raphson.cpp -I/usr/include/python3.12 -lpython3.12 -o Newton_Raphson
./Newton_Raphson

# 3. Rogue-like Room
```

```
g++ -std=c++20 Rl2.cpp -o Rl2
./Rl2

# 4. Tic-Tac-Toe
g++ -std=c++20 tictactoe.cpp -o tictactoe
./tictactoe
```

Listing 1: Bash commands for Task 1 compilation

1.3 Dependencies for Task 2

No entirely new external system libraries are required beyond those already established in Task 1. However, Task 2 extensively utilizes the following features:

- **C++ Standard Library:** Extensive use of `<random>` (specifically `std::mt19937` and `std::uniform_real_distribution`) for synthetic dataset generation, and `<fstream>` for potential data logging.
- **Python 3 & Matplotlib:** This task reuses the `matplotlibcpp.h` wrapper from Task 1 to visualize the multidimensional data and regression lines. As indicated by the executable footprint, the compilation process relies specifically on linking against **Python 3.12** (via `libpython3.12.so`).

1.4 Compilation and Execution Guide (Task 2)

The linear regression engine and dataset generators are heavily modularized into header files (`linearregression.h`, `dataset.h`, `plot.h`, and `coordinate.h`). The entire program is driven by `main.cpp`, which must be compiled with Python linking enabled to support the visualization outputs.

```
# Compile the linear regression program with Python 3.12 linking
# Note: Adjust the include (-I) and link (-l) flags if your local
# Python version differs (e.g., python3.10)
g++ -std=c++20 main.cpp -I/usr/include/python3.12 -lpython3.12 -o
    regression_app

# Execute the program (This will output terminal logs and generate .png
    plots)
./regression_app
```

Listing 2: Bash commands for Task 2 compilation

2 Task 1: Basic C++ Tasks

Although the task called for translating provided python code to C++ we took this opportunity to try and solve the problems ourselves so we wrote all three sections from scratch. All three options were done (one by each of the members of the team).

2.1 Solving Non-linear Equations

This section details the C++ implementation of the Fixed-Point iteration and Newton-Raphson methods. The primary objective was to translate the mathematical logic into robust, reusable C++ code rather than just writing single-use scripts.

Algorithmic Implementation)

Both solvers were designed to be generic. Instead of hardcoding the specific mathematical equations directly inside the solver loops, the `<functional>` library was utilized. By passing the mathematical equations as `std::function<double(double)>` parameters, the solvers act as independent engines capable of evaluating any arbitrary function (e.g., $f(x) = e^{-0.1x}\sin(2x)$ or $f(x) = \cos(x)$) passed from `main()`.

For the Newton-Raphson method, the root-finding formula requires both the function $f(x)$ and its derivative $f'(x)$. Rather than manually deriving and hardcoding a separate derivative function for every test case, the derivative is calculated numerically using the central difference quotient:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

where $h = 0.000001$. This calculation is performed dynamically during each step. Both methods store their iteration history in a `std::list<double>` and terminate when the absolute difference between the current and previous x value falls below a defined tolerance (10^{-5} or 10^{-6}), or when a maximum iteration limit is reached to prevent infinite loops.

Design Choices and Justification

- **Numerical over Analytical Derivatives:** The decision to compute derivatives numerically in the `newton_raphson` function drastically increases the flexibility of the code. While providing an exact analytical derivative might save a marginal amount of CPU time, the numerical approach ensures the solver can handle highly complex functions without requiring the user to manually calculate and write auxiliary derivative functions. It allows the system to adapt to unknown inputs.
- **Data Structure Selection:** `std::list` was initially used to track the iteration history. Since the total number of iterations required to reach convergence is unknown at runtime, a linked list avoids the overhead of memory reallocation that a `std::vector` might experience when growing. However, to interface with the plotting library, this list is converted into a `std::vector` at the end of the execution.

Verification and Visualization

To verify that the code mathematically aligns with the theoretical expectations of the algorithm, `matplotlibcpp.h` was integrated into the Newton-Raphson solver. Upon successfully finding a root, the program automatically generates and saves a convergence plot (`convergence_plot_X.png`). Graphing the iteration number against the x value visually confirms the rapid convergence characteristic of the Newton-Raphson method, as the curve aggressively flattens out precisely at the final calculated root.

2.2 Rouge-Like room moving player

2.2.1 Program Execution and Setup

To compile and execute the navigation module, use a standard C++ compiler such `as g++`. The following commands should be run in the terminal:

1. `g++ RL3.CPP`
2. `./RL3 or ./aout`

2.2.2 Design Rationale: Data Organisation

The core data is organised into custom `struct` types. This was chosen to group related variables togetherlike x and y for the player or height and width for the roommaking the code significantly neater and more structured than using loose variables, i did think of using vectors instead but was like nah structs are easier to understand and maintain.

```
struct room {
    int hieght{};
    int width {};
    std::string ground{"*"};
    std::string corner{"#"};
};

struct player {
    int x{};
    int y{};
    std::string player{"  "};
};
```

Listing 3: Data Structures for Environment and Entities

2.2.3 The Output

Since standard terminals lack an easy "delete" command, a "delightfully devilish" trick was used: printing 100 newlines at the start of each frame to push the previous frame out of view, simulating a screen refresh.

```
// Refresh simulation
for (int j = 0; j < 100; ++j){
    std::cout << '\n';
}

// Logic for drawing the player within the grid
for (int i = 0; i < R.hieght; ++i) {
    for (int j = 0; j < R.width; ++j) {
        if (i == playerpos.x && j == playerpos.y) {
            std::cout << playerpos.player << " ";
        } else {
            std::cout << R.ground << " ";
        }
    }
}
```

Listing 4: Grid Rendering Logic

2.2.4 Movement Logic and Boundary Safety

The movement logic utilises a `switch` statement for efficiency and **pass-by-reference** to ensure the game state actually updates.

```
void movetheplayer(player &playerpos, char input, room room) {
    switch (input) {
        case 'w':
            if (playerpos.x > 0) playerpos.x -= 1;
            break;
        case 's':
            if (playerpos.x < room.hieght - 1) playerpos.x += 1;
            break;
    }
}
```

Listing 5: Movement with Boundary Protection

- **Pass-by-Reference (&):** This gives the function the the actual memory address. Without the `&`, the function would only move a copy of the player, leaving the character on screen stuck.
- **Collision Detection:** The `if` statements act as "boundary protection," ensuring the player cannot walk through the ceilings or walls into invalid memory space.

2.2.5 User Customization

A `do-while` loop was implemented for the room selection menu. This feature ensures the player is trapped in a loop until a valid selection (1, 2, or 3) is made, showing how the rendering engine can dynamically adapt to different scales.

2.3 Tic-Tac-Toe

The final program implemented for Task 1 is a console-based, two-player Tic-Tac-Toe game.

Data Structure and State Management The game board is represented using a two-dimensional vector: `std::vector<vector<char>> grid(3, vector<char>(3, ' '))`. While a 1D array of 9 elements could theoretically store the same data, a 2D vector was deliberately chosen because it inherently maps to the $(row, column)$ coordinate system used by the players. This makes the logic in both the `printBoard` and `winCheck` functions much more intuitive to read and maintain, as the code directly reflects the physical geometry of the game.

The game operates within a continuous `while(true)` loop that handles player turn toggling, grid updates, and standard output rendering. Basic input validation is included to prevent players from overwriting an already occupied cell. If an invalid move is attempted, the program clears the input buffer using `std::cin.clear()` and `std::cin.ignore()` to prevent infinite loops from residual invalid console inputs.

3 Task 2: Linear Regression

This task transitions from basic C++ programming into the fundamental mechanics of machine learning. The objective is to build a prediction model that estimates the relationship between independent explanatory variables (x) and a dependent scalar response (y). Rather than relying on high-level Python libraries like `scikit-learn`, this implementation builds the mathematical engines for both analytical and iterative regression models entirely from scratch in C++.

3.1 Data Generation

Implementation: To test the regression models, a synthetic dataset must first be created. This is handled by the `DataGenerator` class within `dataset.h`. The default C++ `<random>` library was employedspecifically `std::mt19937` (Mersenne Twister) seeded via `std::random_device`. For a given set of hypothetical true weights and an intercept, the generator calculates the perfect y value for an input x, and then adds synthetic noise using `std::uniform_real_distribution`.

Justification: Adding controlled noise to both the x and y variables is a critical step. Without noise, the dataset would form a perfectly straight line, and the regression model would simply be connecting dots. The noise simulates the imperfections of real-world datasets (such as housing market variations). A uniform distribution was specifically chosen to ensure the noise remained bounded and did not look overwhelmingly "wild", guaranteeing the underlying linear trend remained recoverable.

3.2 Simple Linear Regression (Single Variable)

3.2.1 Normal Solution: Ordinary Least Squares (OLS)

The first objective was to implement an analytical solver for simple linear regression. We chose to calculate the required summations ($\sum x, \sum y, \dots$) within a single `for` loop.

```
for (const auto& point : data) {
    sumX += point.x;
    sumY += point.y;
    sumXY += point.x * point.y;
    sumX2 += point.x * point.x;
}
slope = (n * sumXY - sumX * sumY) / (n * sumX2 - sumX * sumX);
```

Listing 6: OLS Summation and Formula Implementation

Rationale and Data Access: This approach is highly efficient for simple tasks. By defining all "sigma" values upfront, we avoid using a redundant number of loops and ensure the computer only parses the dataset once. Using the `Coordinate` struct instead of raw vectors made the code much easier to read; accessing `point.x` and `point.y` is self-documenting compared to using ambiguous array indices.

3.2.2 Iterative Solution: Gradient Descent

For the second objective, we implemented a Gradient Descent algorithm. This approach starts at a neutral base ($w = 0, b = 0$) and iteratively moves toward the best-fit line.

Memory Management and Function Signatures: Efficiency was prioritized by using a specific function signature: `static void performGradientDescent(const std::vector<Coordinate>& data, double& w, double& b...)`

- **Pass-by-Reference (&):** Passing the vector by reference prevents a "deep copy" of the data, which would double memory usage. It gives the function the actual memory address.
- **Const Safety:** The vector is marked `const` to ensure the mathematical solver cannot accidentally modify the source data.
- **Multiple Outputs:** Using references for `w` and `b` allows the function to "return" both the slope and intercept simultaneously without overcomplicating the return types.

The Concept of Epochs and Gradient Resetting: We defined `epochs` as a customizable variable to observe how more loops refine the slope. Inside the outer `epochs` loop, the gradient variables (`dL_dw` and `dL_db`) are reset to zero. This is vital so that errors from previous loops do not "snowball," allowing each iteration to be a clean prediction based only on the current parameters.

Prediction and Batch Processing: Inside each epoch, the computer generates a prediction for every y value.

```
double prediction = (w * point.x) + b;
double error = prediction - point.y;
dL_dw += error * point.x;
dL_db += error;
```

Listing 7: Batch Error Gradient Calculation

By measuring the error for **every single point** before updating the parameters, we perform "Batch" processing. This ensures the line represents the average best fit for the whole group rather than overreacting to individual outliers.

The Update Rule and Learning Rate: The final step updates the weights by subtracting the gradient.

- **Subtraction ($-=$):** Since the gradient points "uphill" toward higher error, we subtract to "descend" toward the minimum.
- **The Learning Rate:** This is the "step size." We found that a high rate (e.g., 10) caused the slope to explode and diverge, while 0.001 provided stable convergence.
- **Normalization ($2.0/n$):** Dividing by the number of points calculates the *mean* gradient, keeping the algorithm stable regardless of how much data is being processed.

3.3 Multiple Linear Regression (Multi variable)

3.3.1 Normal Solution: Ordinary Least Squares (OLS)

To extend the analytical solver to handle multiple variables, the problem must be framed using linear algebra. Instead of calculating simple scalar summations, we construct an augmented matrix representing the system of normal equations ($X^TX|X^TY$).

```
// Populate the XtX (covariance) matrix with nested loops
for (int i = 0; i < k + 1; ++i) {
    for (int j = 0; j < k + 1; ++j) {
        for (int d = 0; d < n; ++d) {
            XtX[i][j] += X[d][i] * X[d][j];
        }
    }
}
// (Followed by Gaussian elimination to solve for w and b)
```

Listing 8: OLS Matrix Population and System Setup

Rationale: This matrix-based approach makes the algorithm highly modular. Rather than hardcoding flat formulas for exactly two variables, dynamically sizing the matrices based on the number of features (k) allows the exact same code to analytically solve for any multidimensional dataset. Gaussian elimination is then used to systematically reduce the augmented matrix and extract the perfect, immediate weights without iterative guessing.

3.3.2 Iterative Solution: Gradient Descent

For the multi-variable Gradient Descent, the core logic remains similar to the single-variable version, but the scalar slope is replaced by a dynamic vector of weights ($w_1, w_2, \ldots, w_k$) to account for every feature's influence on the outcome.

Dynamic Weight Management Instead of a single slope variable, we initialize a `std::vector<double> w` to store the weights and a corresponding `std::vector<double> dL_dw` to track the gradient for each feature. This ensures the model scales automatically with the dataset's dimensionality (k).

Prediction and Error Calculation Inside each epoch, the prediction is now calculated as the sum of the intercept and the dot product of the weights and the input features.

```
double prediction = b;
for (int j = 0; j < k; ++j) {
    prediction += w[j] * point.x[j];
}
double error = prediction - point.y;

for (int j = 0; j < k; ++j) {
    dL_dw[j] += error * point.x[j];
}
dL_db += error;
```

Listing 9: Multi-variable Prediction and Error Gradient

By calculating the error across all k dimensions for **every single point**, the algorithm identifies exactly how much each independent variable contributes to the total error, allowing for precise, multi-directional adjustments.

The Update Rule and Vector Gradients The final step updates the intercept and every weight within the w vector simultaneously.

- **Vectorized Subtraction ($-=$):** A loop iterates through the k features, subtracting the respective feature's gradient from its corresponding weight. This effectively moves the entire multidimensional plane down the gradient slope toward the minimum error.
- **The Learning Rate:** As with the 1D model, the learning rate scales the step size. In a multi-variable space, choosing a small, stable learning rate (0.001) is even more critical. Aggressive steps in multiple dimensions can easily cause the model to overshoot the global minimum and diverge wildly.
- **Normalization ($2.0/n$):** Dividing by the total number of points ensures the mean gradient is used, keeping the descent's stability consistent regardless of how many data points are processed.

3.4 Data Visualization

Implementation & Justification: While the assignment suggested exporting a `.csv` file to plot using Python, a more integrated approach was taken. Using the `matplotlibcpp.h` wrapper, plotting logic was embedded directly into the C++ pipeline (via `plot.h`).

Visualizing the results is the most robust way to conceptually verify the machine learning model. For the 1D regression, the plot cleanly demonstrates both the OLS and GD lines seamlessly bisecting the noisy data cluster. For the 2D multiple regression, visualizing a 3D hyperplane can be difficult to interpret on a 2D screen. Instead, the strategy involved plotting two distinct 2D cross-sections: y versus x_1 (holding x_2 at its mean), and y versus x_2 (holding x_1 at its mean). This design choice isolates the individual impact of each feature, making it explicitly clear how accurately the regression models captured the underlying multidimensional trends.

4 Reflection and Conclusion

This assignment provided a practical look at the differences between high-level scripting in Python and the more manual, structured nature of C++. By building these tools from scratch, we were able to move past using libraries as "black boxes" and actually see how the underlying math and memory management work together.

4.1 Technical Takeaways

- **C++ Structure and Logic:** Moving away from Pythons flexibility required a more disciplined approach to data. Using `structs` to bundle properties and passing by reference (e.g., `player &playerpos`) was essential for keeping the code organized and efficient.

- **Algorithmic Flexibility:** Rather than hardcoding solutions for specific problems, we focused on making our solvers modular. Implementing numerical derivatives for root-finding and matrix-based logic for regression ensured the code could handle different functions or higher-dimensional data without needing to be rewritten.
- **Machine Learning Foundations:** Building the Linear Regression engine manually made the relationship between the loss function and the gradient much clearer. Testing different hyperparameters showed exactly how the learning rate (η) and epoch count directly influence whether a model converges or fails.

4.2 Future Improvements

While the current programs are fully functional and meet all marking criteria, several enhancements would move the code from academic prototypes toward production-quality software[cite: 273, 282]:

- **Efficiency and Robustness:** The Tic-Tac-Toe `winCheck` could be optimized to check only the specific row and column of the last move[cite: 285]. Furthermore, implementing a formal draw state and robust input validation would prevent crashes from non-integer console entries[cite: 285].
- **Scalability:** For Multiple Linear Regression, integrating a dedicated library like **Eigen** would improve matrix-solving stability[cite: 285]. Additionally, implementing an adaptive learning rate would allow the Gradient Descent algorithm to converge more reliably on complex datasets[cite: 178].
- **User Interface:** The Rogue-like module currently uses a "100-newline" trick to simulate a screen refresh; future versions could utilize the `ncurses` library for proper terminal control and smoother navigation.

4.3 Final Summary

Ultimately, this project demonstrated that while the C++ learning curve is steeper, it offers significantly more control over how an algorithm executes. By handling the calculus and linear algebra ourselves, we have a much firmer grasp of how more complex neural networks will function in future work.